

บทที่ 7 การถดถอย (Regression)

วัตถุประสงค์การเรียนรู้

1. อธิบายความหมาย หลักการ และประเภทของการถดถอย (Regression) ในฐานงาน Supervised Learning รวมถึงทฤษฎี Ordinary Least Squares (OLS), Hat Matrix และ Normal Equations
2. สร้างและประเมินโมเดล Simple Linear Regression และ Multiple Regression ด้วย Scikit-learn โดยใช้เมตริก MAE, MSE, RMSE และ R-squared บนชุดข้อมูลจริง
3. ใช้ Polynomial Regression เพื่อจับความสัมพันธ์ที่ไม่เป็นเส้นตรง วิเคราะห์ปัญหา Overfitting จาก Degree สูง และอธิบาย Bias-Variance Tradeoff
4. ประยุกต์ใช้เทคนิค Regularization ได้แก่ Ridge (L2), Lasso (L1) และ Elastic Net เพื่อลดปัญหา Overfitting เลือกค่า α ที่เหมาะสม และเปรียบเทียบผลลัพธ์บนชุดข้อมูล California Housing
5. ดำเนินการ Cross-Validation หลายแบบ ได้แก่ 3-fold, 5-fold, 10-fold และ Leave-One-Out (LOOCV) เพื่อประเมิน Generalization Error และเปรียบเทียบผลระหว่างวิธีต่างๆ
6. เปรียบเทียบวิธี Ensemble ได้แก่ Random Forest (Bagging), Gradient Boosting (Boosting) และ Stacking และเลือกโมเดลที่เหมาะสมที่สุดสำหรับชุดข้อมูล Diabetes และ California Housing โดยอ้างอิงค่า R-squared บน Test Set

7.1 บทนำ

การถดถอย (Regression) เป็นงาน Supervised Learning ที่อัลกอริทึมเรียนรู้เพื่อทำนายค่าผลลัพธ์ต่อเนื่อง (หรือตัวแปรเป้าหมาย) ตาม Feature ที่รับเข้ามา

เป้าหมายคือการค้นหาโมเดลที่ดีที่สุดที่ทำนายค่าเป้าหมายสำหรับข้อมูลใหม่ที่ยังไม่เคยเห็นได้อย่างแม่นยำ

วิธีการ Regression ที่นิยมใช้ใน Scikit-learn มีดังนี้:

7. Linear Regression: โมเดลเรียบง่ายที่ค้นหาความสัมพันธ์เชิงเส้นที่ดีที่สุดระหว่าง Feature และค่าเป้าหมาย
8. Polynomial Regression: ส่วนขยายแบบไม่เชิงเส้นของ Linear Regression ที่ใช้ฟังก์ชัน Polynomial ของ Feature เพื่อ Fit ข้อมูล
9. Ridge Regression: โมเดล Linear Regression ที่รวม Regularization Term เพื่อป้องกัน Overfitting (L2)
10. Lasso Regression: โมเดล Linear Regression ที่รวม Regularization Term เพื่อหาค่า Coefficient ของ Feature ที่ไม่สำคัญให้เป็นศูนย์ (L1)
11. Decision Tree Regression: โมเดลแบบต้นไม้ที่ใช้ชุดกฎ if-then ในการทำนาย
12. Random Forest Regression: วิธี Ensemble ที่รวม Decision Tree หลายต้นเพื่อเพิ่มความแม่นยำ
13. Gradient Boosting Regression: วิธี Ensemble ที่รวมโมเดลอ่อนแอหลายตัวเพื่อเพิ่มความแม่นยำ

7.2 แนวคิดและทฤษฎีที่เกี่ยวข้อง

7.2.1 หลักการ Linear Regression และ Least Squares

Linear Regression เป็นรากฐานของ Machine Learning สำหรับงาน Supervised Learning ที่ output Y เป็นค่าต่อเนื่อง Hastie, Tibshirani และ Friedman (2009) อธิบายว่า โมเดล linear regression มีรูปแบบ $Y = \beta_0 + X_1\beta_1 + \dots + X_p\beta_p + \varepsilon$ เมื่อ $\varepsilon \sim N(0, \sigma^2)$ การ fit โมเดลด้วยวิธี Ordinary Least Squares (OLS) คือการหาค่า $\hat{\beta}$ ที่ minimize Residual Sum of Squares (RSS) ซึ่งนิยามเป็น $RSS(\beta) = (y - X\beta)^T (y - X\beta)$ ซึ่งเป็นฟังก์ชัน quadratic ของพารามิเตอร์ ดังนั้นจึงมี closed-form solution เสมอ

โดยที่:

Y = ตัวแปรตาม (Dependent Variable) ที่ต้องการพยากรณ์

β_0 = ค่าคงที่ (Intercept) จุดที่เส้นถดถอยตัดแกน Y
เมื่อตัวแปรอิสระทุกตัวเป็นศูนย์

$\beta_1, \dots, \beta_p$ = สัมประสิทธิ์การถดถอย (Regression Coefficients)
แสดงผลกระทบของตัวแปรอิสระแต่ละตัว

$X_1, \dots, X_p$ = ตัวแปรอิสระ (Independent Variables / Features)

ε = ค่าความคลาดเคลื่อน (Error Term) ซึ่งสมมติให้มีการแจกแจงปกติ
 $\varepsilon \sim N(0, \sigma^2)$

$RSS(\beta)$ = Residual Sum of Squares ผลรวมกำลังสองของค่าส่วนเหลือ
ใช้เป็นฟังก์ชันที่ต้องทำให้น้อยที่สุด

วิธีหา solution คือการ differentiate $RSS(\beta)$ เทียบกับ β แล้วตั้งให้เท่ากับศูนย์ ผลที่ได้คือ Normal Equations: $X^T(y - X\beta) = 0$ ซึ่งมี unique solution เมื่อ $X^T X$ เป็น positive definite คือ $\hat{\beta} = (X^T X)^{-1} X^T y$ สมการนี้เรียกว่า OLS Estimator และค่าทำนาย $\hat{y} = X\hat{\beta} = X(X^T X)^{-1} X^T y = Hy$ เมื่อ $H = X(X^T X)^{-1} X^T$ เรียกว่า

Hat Matrix เพราะมันใส่ hat ให้ y กล่าวทางเรขาคณิตคือ $\hat{y}$ เป็น orthogonal projection ของ y ลงบน column space ของ X ความแปรปรวนของ OLS estimator คือ $Var(\hat{\beta}) = (X^T X)^{-1} \sigma^2$ ซึ่งแสดงว่า predictor variables ที่มีความแปรปรวนสูงจะให้ coefficient ที่มีความแน่นอนสูงกว่า

โดยที่:

$\beta = (X^T X)^{-1} X^T y$ = สูตรปิด (Closed-Form Solution) ของ Ordinary Least Squares (OLS) ที่ให้ค่า RSS ต่ำที่สุด

X = Design Matrix ขนาด $n \times (p+1)$ บรรจุค่าตัวแปรอิสระทั้งหมด (รวมคอลัมน์ 1 สำหรับ Intercept)

X^T = Transpose ของ X

$(X^T X)^{-1}$ = เมทริกซ์ผกผัน (Inverse Matrix) ต้องมี $X^T X$ ไม่เป็น Singular (ไม่มี Multicollinearity สมบูรณ์)

$\hat{y} = Hy$ = Hat Matrix $H = X(X^T X)^{-1} X^T$ ฉายค่าจริง y ไปยังค่าพยากรณ์ $\hat{y}$ บน Column Space ของ X

7.2.2 เมตริกการประเมินโมเดล Regression

Adjusted R^2

$$\bar{R}^2 = 1 - \frac{(1 - R^2)(n-1)}{n - p - 1}$$

โดยที่:

$\bar{R}^2$ = Adjusted R^2 ที่ปรับตามจำนวนตัวแปรในโมเดล

R^2 = R-Squared เดิม (Coefficient of Determination)

n = จำนวนข้อมูลทั้งหมด

k = จำนวนตัวแปรอิสระในโมเดล

ต่างจาก R^2 ตรงที่ Adjusted R^2 จะลดลงเมื่อเพิ่มตัวแปรที่ไม่มีประโยชน์เข้าไปในโมเดล ช่วยป้องกันการ Overfitting จากการมีตัวแปรมากเกินไป

Akaike Information Criterion (AIC)

$$AIC = 2k - 2\ln \hat{L}$$

โดยที่:

AIC = Akaike Information Criterion ใช้เปรียบเทียบคุณภาพโมเดล

k = จำนวนพารามิเตอร์ในโมเดล

$\ln(\hat{L})$ = Log-Likelihood สูงสุดของโมเดล

หลักการ: โมเดลที่ดีกว่าจะมีค่า AIC ต่ำกว่า AIC สมดุลระหว่างความพอดีของโมเดล (Model Fit) กับความซับซ้อน (Complexity) เพื่อป้องกัน Overfitting

เมตริกหลักในการประเมินโมเดล Regression มีสี่ตัวสำคัญ ได้แก่

Mean Absolute Error (MAE) $= \frac{1}{n} \sum |y_i - \hat{y}_i|$ วัดค่าเฉลี่ยของขนาดความผิดพลาด

มีหน่วยเดียวกับ y ีความได้ง่าย และทนทานต่อ outlier เนื่องจากใช้ค่าสัมบูรณ์แทนกำลังสอง

Mean Absolute Error (MAE):

$$MAE = (1/n) \sum |y_i - \hat{y}_i|$$

โดยที่ y_i คือค่าจริง, $\hat{y}_i$ คือค่าพยากรณ์, n คือจำนวนข้อมูล

MAE

วัดค่าเฉลี่ยของขนาดความผิดพลาดโดยไม่ลงโทษความผิดพลาดขนาดใหญ่เป็นพิเศษ

ีความได้ง่ายเพราะมีหน่วยเดียวกับตัวแปรตาม และทนทานต่อค่าผิดปกติ (Outlier)

มากกว่า MSE

Mean Squared Error (MSE) $= \frac{1}{n} \sum (y_i - \hat{y}_i)^2$

ลงโทษความผิดพลาดขนาดใหญ่หนักกว่า MAE เนื่องจากใช้กำลังสอง

มีหน่วยเป็นกำลังสองของ y ดังนั้นจึงยากต่อการตีความโดยตรง แต่มีคุณลักษณะทางคณิตศาสตร์ที่ดีกว่าในการ optimize

Mean Squared Error (MSE):

$$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$$

โดยที่ y_i คือค่าจริง, $\hat{y}_i$ คือค่าพยากรณ์, n คือจำนวนข้อมูล

MSE ยกกำลังสองของความผิดพลาด ทำให้ลงโทษความผิดพลาดขนาดใหญ่หนักกว่า MAE

ข้อเสีย: หน่วยเป็นกำลังสอง ตีความตรงๆ ได้ยาก จึงมักใช้ RMSE แทนในการรายงาน

Root Mean Squared Error (RMSE) $= \sqrt{MSE}$ รากที่สองของ MSE ทำให้หน่วยกลับมาเหมือน y ผสมผสานความไวต่อ outlier ของ MSE กับความสามารถในการตีความของ MAE จึงนิยมใช้มากที่สุดในทางปฏิบัติ

Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{MSE} = \sqrt{(1/n) \sum (y_i - \hat{y}_i)^2}$$

RMSE คือรากที่สองของ MSE ทำให้หน่วยกลับมาเป็นหน่วยเดียวกับตัวแปรตาม

นิยมใช้รายงานผลโมเดล Regression เพราะตีความได้ง่ายกว่า MSE

R-squared	(R²)	$= 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$
------------------	------------------------	---

วัดสัดส่วนของความแปรปรวนในข้อมูลที่โมเดลสามารถอธิบายได้ มีค่าระหว่าง 0 ถึง 1 (สำหรับ OLS บน training set) โดย 1 หมายถึงโมเดลอธิบายข้อมูลได้สมบูรณ์ และ 0 หมายถึงโมเดลไม่ดีกว่าการใช้ค่าเฉลี่ยเป็นค่าทำนาย บน test set R² อาจมีค่าติดลบหากโมเดลแย่กว่าค่าเฉลี่ย

R² (Coefficient of Determination):

$$R^2 = 1 - SS_{res}/SS_{tot} = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$$

โดยที่ SS_{res} = ผลรวมกำลังสองของส่วนเหลือ (Residual), SS_{tot} =
ผลรวมกำลังสองรวม

ช่วง: $R^2 \in [0, 1]$ ยิ่งใกล้ 1 โมเดลอธิบายความแปรปรวนของ Y ได้มากขึ้น

ข้อควรระวัง: R^2 ไม่ลดลงเมื่อเพิ่มตัวแปร จึงต้องใช้ Adjusted R^2

เปรียบเทียบโมเดลที่มีจำนวนตัวแปรต่างกัน

7.2.3 Polynomial Regression และปัญหา Overfitting

Polynomial Regression เป็นการขยาย Linear Regression โดยสร้าง features ใหม่จากกำลังของ features เดิม: $x, x^2, x^3, \dots, x^m$ ทำให้สามารถจับความสัมพันธ์ที่ไม่เป็นเส้นตรงได้ ในแง่ทฤษฎีโมเดลยังคงเป็น linear ใน parameters β แต่ไม่ linear ใน input x ดังนั้น OLS ยังสามารถใช้หา $\hat{\beta}$ ได้เหมือนเดิม อย่างไรก็ตาม degree สูงนำไปสู่ปัญหา Overfitting กล่าวคือโมเดลจะ fit ข้อมูล training ได้ดีมาก (training R^2 สูง) แต่ generalize ไปยัง test data ได้ไม่ดี เนื่องจากโมเดลเรียนรู้ noise ในข้อมูล training แทนที่จะเรียนรู้ pattern ที่แท้จริง ปรากฏการณ์นี้เรียกว่า Bias-Variance Tradeoff โดยโมเดลที่ซับซ้อนมี low bias แต่ high variance

7.2.4 ทฤษฎี Regularization: Ridge, Lasso และ Elastic Net

Gradient Descent Update Rule

$$\theta_j \leftarrow \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

โดยที่:

θ_j = พารามิเตอร์ j ของโมเดล (เช่น ค่าสัมประสิทธิ์)

α = Learning Rate ควบคุมขนาดก้าวการอัปเดต

$\partial J / \partial \theta_j$ = Gradient ของฟังก์ชันค่าใช้จ่าย J เทียบกับ θ_j

อัลกอริทึม Gradient Descent อัปเดตพารามิเตอร์ในทิศทางตรงข้ามกับ Gradient ซ้ำๆ จนกว่าจะบรรจบ (Converge) สู่ค่าต่ำสุด

Variance Inflation Factor (VIF)

$$VIF_j = \frac{1}{1 - R_j^2}$$

โดยที่:

VIF_j = Variance Inflation Factor ของตัวแปร j

R_j^2 = R^2 จากการถดถอยตัวแปร j กับตัวแปรอิสระตัวอื่นๆ ทั้งหมด

เกณฑ์: $VIF < 5$ ยอมรับได้, $VIF 5-10$ น่ากังวล, $VIF > 10$ แสดงถึงปัญหา

Multicollinearity รุนแรงที่ต้องแก้ไขก่อนสร้างโมเดล

Regularization คือการเพิ่ม penalty term เข้าไปใน objective function เพื่อลด coefficient ให้มีขนาดเล็กลง ซึ่งป้องกัน Overfitting โดยลด variance ของโมเดลแลกกับการยอมรับ bias เพิ่มขึ้นเล็กน้อย Hastie et al. (2009) นำเสนอ Shrinkage Methods ไว้ดังนี้

Ridge Regression (L2 Regularization) minimize $RSS(\beta) + \lambda \sum_{j=1}^p \beta_j^2$ ซึ่งมี closed-

form solution คือ $\hat{\beta}^{ridge} = (X^T X + \lambda I)^{-1} X^T y$ การเพิ่ม λ เข้าไปในเมทริกซ์ทำให้ปัญหา ill-conditioned (ซึ่งเกิดจาก multicollinearity) แก้ไขได้ด้วย λ ที่ทำให้เมทริกซ์กลับได้เสมอ Ridge “หด” coefficients ทุกตัวเข้าหาศูนย์ แต่ไม่ทำให้เป็นศูนย์พอดี เหมาะสำหรับกรณีที่ features ทุกตัวมีส่วนสำคัญในการทำนาย ค่า λ ที่ใหญ่ขึ้นทำให้ coefficients หดลงมากขึ้น

Ridge Regression (L2 Regularization):

$$\text{minimize: } RSS(\beta) + \lambda \sum_j \beta_j^2$$

$$\text{สูตรปิด: } \beta_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

โดยที่:

λ (Lambda) = Hyperparameter ควบคุมความแรงของ Regularization ($\lambda=0$ คือ OLS ปกติ)

$\Sigma \beta_j^2$ = L2 Penalty บีบสัมประสิทธิ์ทุกตัวให้เข้าใกล้ศูนย์แต่ไม่เป็นศูนย์พอดี

λ = เมทริกซ์ Identity คูณ λ ช่วยให้ $X^T X + \lambda I$ มี Inverse เสมอแม้มี Multicollinearity

ผล: ลด Variance ของโมเดลแลกกับ Bias เล็กน้อย ป้องกัน Overfitting

Lasso Regression (L1 Regularization) minimize $\frac{1}{2} RSS(\beta) + \lambda \sum_{j=1}^p |\beta_j|$

ข้อแตกต่างสำคัญจาก Ridge คือการใช้ L1 norm (ค่าสัมบูรณ์) แทน L2 norm (กำลังสอง) ทำให้ Lasso มีคุณลักษณะพิเศษที่ทำให้ coefficients ของ features ที่ไม่สำคัญกลายเป็น 0 พอดี (sparse solution) ซึ่งเทียบเท่ากับการทำ Feature Selection อัตโนมัติ เนื่องจาก Lasso ไม่มี closed-form solution เหมือน Ridge จึงต้องใช้ iterative algorithms เช่น Coordinate Descent ในทางเรขาคณิต constraint ของ Lasso เป็นรูปทรงหลายเหลี่ยม (diamond) ที่มีมุมแหลม ทำให้ solution มักตกที่มุมซึ่งหมายถึง coefficient บางตัวเป็น 0

Lasso Regression (L1 Regularization):

minimize: $(1/2) RSS(\beta) + \lambda \sum_j |\beta_j|$

โดยที่:

$\sum |\beta_j|$ = L1 Penalty ใช้ค่าสัมบูรณ์ของสัมประสิทธิ์

ข้อแตกต่างจาก Ridge: L1 Penalty สามารถทำให้สัมประสิทธิ์บางตัวเป็นศูนย์พอดี

ผล: Lasso ทำ Feature Selection อัตโนมัติ

เหมาะกับข้อมูลที่มีตัวแปรไม่เกี่ยวข้องจำนวนมาก

ข้อจำกัด: ไม่มีสูตรปิด ต้องใช้ Coordinate Descent ในการหาคำตอบ

Elastic Net รวม L1 และ L2 penalty เข้าด้วยกัน: minimize เมื่อ α คือ l1_ratio ที่ควบคุมสัดส่วนระหว่าง L1 และ L2 ข้อดีคือได้ประโยชน์จากทั้งสองแบบ ได้แก่ Feature Selection จาก L1 และความเสถียรเมื่อ features มี correlation สูงจาก L2 โดยทั่วไปใน

scikit-learn พารามิเตอร์ α สอดคล้องกับ λ ดังนั้น α สูง = regularization สูง = โมเดลเรียบง่ายขึ้น

7.2.5 ทฤษฎี Cross-Validation

Cross-Validation เป็นเทคนิคประมาณ generalization error ของโมเดล ซึ่งสำคัญมากเพราะ training error มักต่ำกว่า test error อยู่เสมอ Hastie et al. (2009) อธิบาย K-fold Cross-Validation ว่าทำงานโดยแบ่งชุดข้อมูลออกเป็น K กลุ่มย่อย (folds) ที่มีขนาดเท่ากัน จากนั้นวนซ้ำ K รอบ ในแต่ละรอบใช้ข้อมูล K-1 กลุ่มสำหรับ training และ 1 กลุ่มที่เหลือสำหรับ validation สุดท้ายนำค่า error ทั้ง K รอบมาเฉลี่ยกัน

การเลือกค่า K มี trade-off ที่สำคัญ ค่า K ต่ำ (เช่น K=3) ทำให้ training set มีขนาดเล็กกว่า อาจ overestimate error แต่ใช้เวลาคำนวณน้อย ค่า K สูง (เช่น K=10) ให้การประมาณที่แม่นยำกว่าเพราะ training set ใกล้เคียงกับข้อมูลทั้งหมดมากขึ้น Leave-One-Out Cross-Validation (LOOCV) เป็นกรณีพิเศษที่ K=n ทุก data point จะถูกใช้เป็น validation set หนึ่งครั้ง LOOCV ให้การประมาณที่ unbiased ที่สุด แต่ใช้เวลาคำนวณมากที่สุดและมี high variance ในการประมาณ ค่า K=5 หรือ K=10 มักเป็นทางเลือกที่ดีในทางปฏิบัติ

7.2.6 ทฤษฎี Ensemble Methods: Bagging, Boosting และ Stacking

Ensemble Methods อาศัยหลักการรวมโมเดลหลายตัวเพื่อให้ได้ผลลัพธ์ที่ดีกว่าโมเดลเดี่ยว หลักการเบื้องต้นหลังคือความผิดพลาดของโมเดลแต่ละตัวมักไม่สัมพันธ์กัน ทำให้เมื่อรวมกันแล้วความผิดพลาดจะหักล้างกัน

Bagging (Bootstrap Aggregating) Hastie et al. (2009) กำหนด Bagging estimate เป็น
$$\hat{f}^{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$
 เมื่อ $\hat{f}^{*b}(x)$ คือการทำนายของโมเดลที่ฝึกบน bootstrap sample ที่ b ด้วยการสุ่ม bootstrap B ชุด ฝึกโมเดลแยกกันทุกชุด แล้วเฉลี่ยผลลัพธ์ (regression) หรือ majority vote (classification) Bagging ลด variance ของโมเดลที่

unstable เช่น Decision Tree ได้อย่างมีประสิทธิภาพ เนื่องจากค่าเฉลี่ยของ B independent variables แต่ละตัวมี variance σ^2/B (ลดลง B เท่า) Random Forest เป็น Bagging ที่พัฒนาต่อโดยเพิ่มการสุ่ม features ในแต่ละ split เพื่อลด correlation ระหว่างต้นไม้

Bootstrap Aggregating (Bagging):

$$f_{\text{bag}}(x) = (1/B) \sum_{\beta} f_{\beta}^*(x)$$

โดยที่:

B = จำนวนโมเดลย่อย (Base Learners) ที่สร้างขึ้น

$f_{\beta}^*(x)$ = โมเดลที่ b ฝึกบน Bootstrap Sample ชุดที่ b (สุ่มข้อมูลแบบ With Replacement)

$f_{\text{bag}}(x)$ = ค่าเฉลี่ยของผลลัพธ์จากโมเดลทุกตัว (สำหรับ Regression)

หลักการ: การเฉลี่ยผลจากโมเดลหลายตัวช่วยลด Variance โดยไม่เพิ่ม Bias มากนัก

Random Forest คือ Bagging ของ Decision Tree + สุ่ม Feature Subset
ทุกการแบ่ง

Boosting ต่างจาก Bagging ตรงที่สร้างโมเดลแบบ sequential ไม่ใช่ parallel แต่ละโมเดลถัดไปมุ่งแก้ไขความผิดพลาดที่โมเดลก่อนหน้าทำ ใน Gradient Boosting แต่ละ iteration fit tree ลง gradient ของ loss function ผลลัพธ์สุดท้ายคือ weighted sum ของ weak learners ที่รวมกันเป็น strong learner Boosting ลด bias ของโมเดลเป็นหลัก (ต่างจาก Bagging ที่ลด variance) ข้อควรระวังคือ Boosting อาจ overfit ได้หากจำนวน iterations มากเกินไป

Stacking ใช้โมเดลระดับที่สอง (Meta-learner) เพื่อรวมผลลัพธ์ของโมเดลระดับแรก (Base learners) โดย Meta-learner เรียนรู้ว่าควรให้น้ำหนักกับผลลัพธ์ของ Base learner แต่ละตัวเท่าใด แทนที่จะใช้การเฉลี่ยแบบง่าย Stacking มีความยืดหยุ่นสูงกว่า Bagging และ Boosting เพราะ Base learners ไม่จำเป็นต้องเป็นโมเดลประเภทเดียวกัน

ตารางที่ 7.1 สรุปเปรียบเทียบทฤษฎีและลักษณะสำคัญของวิธีการ Regression และ Ensemble ในบทนี้

วิธีการ	Objective Function	ลด Variance/Bias	ลักษณะพิเศษ
OLS	$minimize\ RSS(\boldsymbol{\beta})$	ไม่ลด	Closed-form solution
Ridge (L2)	$RSS(\boldsymbol{\beta}) + \lambda \sum \beta_j^2$	ลด Variance	หัด coeff ทุกตัวไม่เป็น 0
Lasso (L1)	$RSS(\boldsymbol{\beta}) + \lambda \sum \beta_j $	ลด Variance	Feature Selection อัตโนมัติ
Elastic Net	$RSS(\boldsymbol{\beta}) + \lambda \left[\alpha \sum \beta_j + (1-\alpha) \sum \beta_j^2 \right]$ Elastic Net Regularization: minimize: $RSS(\boldsymbol{\beta}) + \lambda [\alpha \sum \beta_j + (1-\alpha) \sum \beta_j^2]$ โดยที่: λ (Lambda) = Hyperparameter ควบคุมความแรงของ Regularization รวม α (Alpha) = Mixing Parameter $\in [0, 1]$ ควบคุมสัดส่วน L1 ต่อ L2	ลด Variance	รวมข้อดีของ L1+L2

วิธีการ	Objective Function	ลด Variance/Bias	ลักษณะพิเศษ
	$\alpha = 1$ คือ Lasso, $\alpha = 0$ คือ Ridge ข้อดี: รวมจุดแข็งของ Ridge (รับมือ Multicollinearity) และ Lasso (Feature Selection) เหมาะกับข้อมูลที่มีตัวแปรจำนวนมากและอาจมีกลุ่มตัวแปรที่สัมพันธ์กัน		
Bagging/Random Forest	$\hat{f}^{bag}(x) = (1/B) \sum \hat{f}^b(x)$	ลด Variance หลัก	Parallel, Bootstrap sampling
Gradient Boosting	Fit residuals ของโมเดลก่อนหน้า	ลด Bias หลัก	Sequential, อาจ Overfit

ที่มา: ปรับปรุงจาก Hastie, Tibshirani, and Friedman (2009), *The Elements of Statistical Learning*, Chapters 3 และ 8

7.3 Simple Regression

Simple Regression หรือ Linear Regression เป็นเทคนิคสถิติและ Machine Learning พื้นฐานที่ใช้สำหรับการสร้างโมเดลและวิเคราะห์ความสัมพันธ์เชิงเส้นระหว่างสองตัวแปร ได้แก่ ตัวแปรอิสระ (Independent Variable) หนึ่งตัวและตัวแปรตาม (Dependent Variable) หนึ่งตัว

เมตริกสำหรับประเมินโมเดล Regression ได้แก่ MAE (Mean Absolute Error), MSE (Mean Squared Error), RMSE (Root Mean Squared Error) และ R-squared (R^2) ซึ่งวัดสัดส่วนความแปรปรวนที่โมเดลอธิบายได้

7.3.1 แบบฝึกหัด – ราคาที่อยู่อาศัยในแคลิฟอร์เนีย (Simple Regression)

เอกสารอ้างอิง:

[https://scikit-](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html)

[learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html)

7.3.1.1 การตั้งค่า

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
df = pd.read_csv('/content/sample_data/california_housing_train.csv')
```

7.3.1.2 ทดลอง: total_rooms กับ total_bedrooms

เตรียมตัวแปรอิสระและตัวแปรตาม จากนั้นแบ่งข้อมูลและสร้างโมเดล:

```
x = np.array(df['total_rooms']).reshape(-1,1)
y = np.array(df['total_bedrooms']).reshape(-1,1)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.20)
reg = LinearRegression()
model = reg.fit(x_train, y_train)
print(model.coef_, model.intercept_) #
[[0.181]], [61.87]

y_pred = model.predict(x_test)
print('MAE:', mean_absolute_error(y_test, y_pred)) # 101.36
print('RMSE:', mean_squared_error(y_test, y_pred, squared=False)) # 154.12
print('R^2:', r2_score(y_test, y_pred)) # 0.864
```

ผลลัพธ์: $R^2 = 0.864$ แสดงว่าโมเดลอธิบายความแปรปรวนของข้อมูลได้ถึง 86.4%

7.3.1.3 ทดลอง: median_income กับ median_house_value

```
x = np.array(df['median_income']).reshape(-1,1)
y = np.array(df['median_house_value']).reshape(-1,1)
```

ผลลัพธ์: $R^2 = 0.494$ — Coefficient = [41,854.98], Intercept = [44,955.95] แสดงว่า Income ที่เพิ่มขึ้น 1 หน่วยสัมพันธ์กับราคาบ้านที่เพิ่มขึ้นประมาณ 41,855 ดอลลาร์

7.3.1.4 ทดลอง: households กับ population

```
x = np.array(df['households']).reshape(-1,1)
y = np.array(df['population']).reshape(-1,1)
```

ผลลัพธ์: $R^2 = 0.844$ — Coefficient = [2.74], Intercept = [55.92]
แสดงความสัมพันธ์เชิงบวกที่แข็งแกร่งระหว่างจำนวนครัวเรือนและประชากร

7.3.2 แบบฝึกหัด – Polynomial Regression

เอกสารอ้างอิง: https://scikit-learn.org/stable/modules/linear_model.html#polynomial-regression

7.3.2.1 ทดลองกับ Simple Linear Regression ก่อน

สร้างข้อมูลสังเคราะห์ที่มีความสัมพันธ์แบบกำลังสอง: $y = x^2 - x - 1 + \text{noise}$

```
X = np.random.uniform(-3, 3, 100).reshape(-1,1)
y = X*X - X - 1 + np.random.rand(100).reshape(-1,1)*2
model = LinearRegression().fit(X_train, y_train)
```

ผลลัพธ์ Linear Regression: $R^2 = 0.091$ — แย่มาก เพราะข้อมูลมีความสัมพันธ์แบบ Non-Linear

7.3.2.2 ใช้ Polynomial Features

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
poly_train = poly.fit_transform(x_train.reshape(-1,1))
poly_test = poly.fit_transform(x_test.reshape(-1,1))
poly_reg_model = LinearRegression()
poly_reg_model.fit(poly_train, y_train)
```

ผลลัพธ์ degree=2: $R^2 = 0.960$ — ดีขึ้นมากเพราะ Feature แบบกำลังสองตรงกับลักษณะข้อมูล

7.3.2.3 Degree สูงเกินไป → Overfitting

ทดลอง degree=51: Training $R^2 = 0.840$ แต่ Testing $R^2 = 0.123$ — แย่มาก!
Polynomial Degree สูงทำให้ Overfit กับข้อมูล Train แต่ทำนายข้อมูลใหม่ได้ไม่ดี

7.4 Multiple Regression

Multiple Regression เป็นเทคนิคสถิติ และ Machine Learning ที่ทรงพลังสำหรับการสร้างโมเดลความสัมพันธ์ระหว่างตัวแปรอิสระหลายตัว (Predictors) กับตัวแปรตามหนึ่งตัว ช่วยให้เข้าใจว่าการเปลี่ยนแปลงในหลายตัวแปรส่งผลต่อผลลัพธ์เดียวอย่างไร

7.4.1 แบบฝึกหัด – ราคาที่อยู่อาศัยในแคลิฟอร์เนีย (Multiple Regression)

7.4.1.1 ทดลองด้วย 2 ตัวแปรอิสระ

ใช้ total_rooms และ median_income ทำนาย total_bedrooms:

```
X = np.array(df[['total_rooms',  
                'median_income']]).reshape(-1,2)  
y = np.array(df['total_bedrooms']).reshape(-1,1)  
model = LinearRegression().fit(X_train, y_train)  
print(model.coef_, model.intercept_) # [[0.187,  
-44.68]], [218.64]
```

ผลลัพธ์: $R^2 = 0.896$ — ดีกว่า Simple Regression (0.864) เพราะเพิ่ม median_income เข้ามาช่วย

7.4.1.2 ทดลองด้วย 6 ตัวแปรอิสระ

เพิ่ม housing_median_age, total_rooms, population, households,
median_income, median_house_value:

```
X = np.array(df[['housing_median_age', 'total_rooms', 'population', 'households', 'median_income', 'median_house_value']]).reshape(-1, 6)
```

ผลลัพธ์: $R^2 = 0.975$ — ยังมีตัวแปรมาก ความแม่นยำยิ่งเพิ่มขึ้นอย่างมีนัยสำคัญ

7.5 Regularization

Regularization เป็นเทคนิคสำคัญใน Machine Learning ที่ช่วยป้องกัน Overfitting — ปัญหาที่โมเดลเรียนรู้ข้อมูล Train ดีเกินไปแต่ไม่สามารถ Generalize กับข้อมูลใหม่ได้ Regularization เพิ่ม Constraint ให้กับโมเดลเพื่อลด Overfitting

Scikit-learn รองรับ 3 เทคนิค Regularization หลัก:

14. Ridge (L2 Regularization): เพิ่มพจน์ลงโทษ (Penalty Term) เป็นผลรวมกำลังสองของ Coefficient ควบคุมด้วย alpha
15. Lasso (L1 Regularization): เพิ่ม Penalty Term เป็นผลรวมค่าสัมบูรณ์ของ Coefficient สามารถทำให้ Coefficient บางตัวเป็นศูนย์ได้ (Feature Selection อัตโนมัติ)
16. Elastic Net: รวม L1 และ L2 Regularization เข้าด้วยกัน ควบคุมด้วย alpha และ l1_ratio

7.5.1 แบบฝึกหัด – Regularization

ใช้ข้อมูลสังเคราะห์ $y = -5X + X^2 + \text{noise}$ ทดสอบเทคนิคต่างๆ:

7.5.1.1 Linear Regression (Baseline)

ผลลัพธ์: Training $R^2 = 0.0008$, Testing $R^2 = -0.0453$ — แย่มาก ข้อมูลไม่เป็นเส้นตรง

7.5.1.2 Polynomial Regression (degree=2)

```
from sklearn.linear_model import
LinearRegression, Ridge, Lasso, ElasticNet

from sklearn.preprocessing import
PolynomialFeatures

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)
```

ผลลัพธ์: Training $R^2 = 0.953$, Testing $R^2 = 0.923$ — ดีมาก!

7.5.1.3 Polynomial Degree สูง (degree=21) → Overfitting

ผลลัพธ์: Training $R^2 = 0.962$ (ดีกว่า degree=2) แต่ Testing $R^2 = 0.910$ (แยกลง) — Overfitting!

7.5.1.4 Regularization (Ridge, Lasso, Elastic Net)

```
alpha_ridge = alpha_lasso = alpha_elasticnet =
0.001

ridge_model =
Ridge(alpha=alpha_ridge).fit(X_poly_high_degree
_train, y_train)

lasso_model =
Lasso(alpha=alpha_lasso).fit(X_poly_high_degree
_train, y_train)

elasticnet_model =
ElasticNet(alpha=alpha_elasticnet,
l1_ratio=0.5).fit(X_poly_high_degree_train,
y_train)
```

ผลลัพธ์: Ridge $R^2 = 0.963/0.915$, Lasso $R^2 = 0.955/0.923$, ElasticNet $R^2 = 0.955/0.923$ — Regularization ช่วยลด Gap ระหว่าง Train และ Test ได้ดี

7.5.2 กรณีศึกษา – ราคาที่อยู่อาศัยในแคลิฟอร์เนีย (Regularization)

ใช้ชุดข้อมูล California Housing Prices จาก Scikit-learn ทดสอบ Ridge, Lasso และ Elastic Net กับ alpha หลายค่า: [0.001, 0.01, 0.1, 1, 10, 50]

```
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
data = fetch_california_housing()
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

ผลลัพธ์สรุป: Linear Regression (Baseline): $R^2 = 0.613/0.576$ / Ridge (alpha=50): R^2 ดีที่สุดที่ 0.667 / Lasso (alpha=0.001): $R^2 = 0.669$ (ดีที่สุด) / ElasticNet (alpha=0.001, l1_ratio=0.9): $R^2 = 0.669$

7.6 Cross-Validation

Cross-Validation

เป็นเทคนิคที่ใช้ประมาณว่าโมเดลจะทำงานได้ดีเพียงใดกับข้อมูลที่ยังไม่เคยเห็น โดยการแบ่งชุดข้อมูลออกเป็นหลาย Subset อย่างเป็นระบบ ช่วยลดปัญหา Overfitting และให้การประเมินประสิทธิภาพโมเดลที่แม่นยำยิ่งขึ้น Scikit-learn รองรับ KFold, StratifiedKFold และ LeaveOneOut

7.6.1 แบบฝึกหัด – Cross-Validation

ทดสอบด้วยข้อมูลสังเคราะห์: $y = X + \text{noise}$ โดยใช้ 4 เทคนิค Cross-Validation

```
from sklearn.model_selection import KFold, LeaveOneOut

from sklearn.linear_model import LinearRegression

from sklearn.model_selection import KFold, LeaveOneOut, cross_val_score
```

```
import numpy as np

# สร้างข้อมูลสังเคราะห์  $y = X + \text{noise}$ 
np.random.seed(42)
X_cv = np.random.uniform(0, 10, 100).reshape(-1, 1)
y_cv = X_cv.ravel() + np.random.normal(0, 1, 100)
model_cv = LinearRegression()

cv_configs = [
    ("3-fold", KFold(n_splits=3, shuffle=True, random_state=0)),
    ("5-fold", KFold(n_splits=5, shuffle=True, random_state=0)),
    ("10-fold", KFold(n_splits=10, shuffle=True, random_state=0)),
    ("Leave-One-Out", LeaveOneOut()),
]

for name, cv in cv_configs:
    scores = cross_val_score(model_cv, X_cv, y_cv, cv=cv,
                              scoring="neg_mean_squared_error")
    mse = -scores.mean()
    print(f"{name:15s} MSE = {mse:.4f}")
```

ผลลัพธ์: 3-fold MSE = 1.065 / 5-fold MSE = 1.084 / 10-fold MSE = 1.057 / LOO MSE = 1.051 — Leave-One-Out ให้ MSE ต่ำที่สุด แต่ใช้เวลาคำนวณมากที่สุด

7.6.2 กรณีศึกษา – ราคาที่อยู่อาศัยในแคลิฟอร์เนีย (Cross-Validation)

ใช้ชุดข้อมูล California Housing Prices จาก Scikit-learn ทดสอบ Cross-Validation ทั้ง 4 แบบ

```
from sklearn.datasets import
fetch_california_housing
data = fetch_california_housing()
```

ผลลัพธ์: 3-fold MSE = 0.526 / 5-fold MSE = 0.528 / 10-fold MSE = 0.528 / LOO MSE = 0.528 — ทุกวิธีให้ผลใกล้เคียงกัน แสดงว่าโมเดล Stable ดี

7.7 Ensemble Methods

Ensemble Methods เป็นเทคนิคที่ทรงพลังใน Machine Learning ที่รวมการทำนายของโมเดลหลายตัวเพื่อเพิ่มประสิทธิภาพโดยรวม อาศัยหลักการ 'ความฉลาดของฝูงชน' โดยเฉพาะมีประสิทธิภาพในการลด Overfitting และเพิ่ม Generalization

ประเภทของ Ensemble Methods:

17. Bagging: สร้างโมเดลฐานหลายโมเดลขนาน แต่ละตัวฝึกด้วย Subset ของข้อมูลที่แตกต่างกัน ตัวอย่าง: Random Forest
18. Boosting: สร้างโมเดลฐานแบบลำดับ แต่ละโมเดลมุ่งเน้นข้อผิดพลาดของโมเดลก่อนหน้า ตัวอย่าง: Gradient Boosting
19. Stacking: เทคนิค Ensemble ขั้นสูงที่รวมการทำนายของโมเดลฐานหลายตัวโดยใช้ Meta-learner

7.7.1 แบบฝึกหัด – การจำแนก Iris แบบ Binary ด้วย Random Forest

เอกสารอ้างอิง : <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

```
from sklearn.ensemble import
RandomForestClassifier

classifier =
RandomForestClassifier(n_estimators=10)

classifier.fit(X_train, y_train)

print('Accuracy:', accuracy_score(y_test,
classifier.predict(X_test))) # 0.95
```

7.7.2 แบบฝึกหัด – การจำแนก Iris แบบ Multiclass ด้วย Random Forest

ใช้ Iris ทั้ง 3 คลาส ด้วย $n_estimators=10$ ผลลัพธ์: Accuracy = 0.967 — Random Forest ให้ผลดีกว่า Decision Tree เดียวๆ

7.7.3 กรณีศึกษา – ราคาที่อยู่อาศัยในแคลิฟอร์เนีย (Ensemble Methods)

เปรียบเทียบโมเดล Regression หลายประเภทบนชุดข้อมูล California Housing Prices:

7.7.3.1 Linear Regression (Baseline)

Training $R^2 = 0.613$, Testing $R^2 = 0.576$

7.7.3.2 Polynomial Regression (degree=2)

Training $R^2 = 0.685$, Testing $R^2 = 0.646$ — ดีขึ้นเล็กน้อย Degree 3+ ทำให้ Overfit

7.7.3.3 Regularization (Ridge/Lasso/ElasticNet)

Ridge best ($\alpha=50$): Testing $R^2 = 0.667$ / Lasso best ($\alpha=0.001$): Testing $R^2 = 0.669$ / ElasticNet best: Testing $R^2 = 0.669$

7.7.3.4 Multivariable Regression

Training $R^2 = 0.613$, Testing $R^2 = 0.576$ — เท่ากับ Linear Regression
เพราะใช้ฟังก์ชันเดียวกัน

7.7.3.5 Random Forest Regression (Bagging)

```
from sklearn.ensemble import
RandomForestRegressor

rf_model =
RandomForestRegressor(n_estimators=100,
random_state=42)

rf_model.fit(X_train_scaled, y_train)
```

ผลลัพธ์: Training $R^2 = 0.974$ (สูงมาก!), Testing $R^2 = 0.805$ — Random Forest
ให้ผลดีที่สุดในทุกวิธี

7.7.3.6 Gradient Boosting Regression (Boosting)

```
from sklearn.ensemble import
GradientBoostingRegressor

gb_model =
GradientBoostingRegressor(n_estimators=200,
learning_rate=0.1,
max_depth=3,
random_state=42)

gb_model.fit(X_train_scaled, y_train)

print("Gradient Boosting Training  $R^2$ :",
gb_model.score(X_train_scaled, y_train))

print("Gradient Boosting Testing  $R^2$ :",
gb_model.score(X_test_scaled, y_test))
```


ผลลัพธ์: Training $R^2 = 0.869$, Testing $R^2 = 0.836$ — Gradient Boosting ให้ผลดีกว่า Random Forest เล็กน้อย เพราะสร้าง Tree แบบ Sequential มุ่งแก้ข้อผิดพลาดของ Tree ก่อนหน้า

7.7.3.7 Stacking Regression

```
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import Ridge

# Base learners: Random Forest + Gradient Boosting
estimators = [
    ("rf", RandomForestRegressor(n_estimators=100,
                                random_state=42)),
    ("gb", GradientBoostingRegressor(n_estimators=200,
                                     learning_rate=0.1,
                                     max_depth=3, random_state=42)),
]

# Meta-learner: Ridge
stacking_model = StackingRegressor(estimators=estimators,
                                   final_estimator=Ridge(),
                                   cv=5)
stacking_model.fit(X_train_scaled, y_train)

print("Stacking Training  $R^2$ :",
      stacking_model.score(X_train_scaled, y_train))
print("Stacking Testing  $R^2$ :",
      stacking_model.score(X_test_scaled, y_test))
```

ผลลัพธ์: Training $R^2 = 0.951$, Testing $R^2 = 0.839$ — Stacking รวมจุดแข็งของ Random Forest (Bagging) และ Gradient Boosting (Boosting) โดยให้ Ridge เป็น Meta-learner กำหนดน้ำหนักที่เหมาะสม

7.7.3.6 สรุปการเปรียบเทียบ

Random Forest (Bagging) ให้ Testing R^2 สูงสุดที่ 0.805 ตามด้วย Polynomial + Regularization ที่ 0.669 ส่วน Linear Regression พื้นฐานให้ 0.576 — Ensemble Methods มีประสิทธิภาพสูงกว่าอย่างชัดเจน

7.8 การเปรียบเทียบวิธีการ Regression

ส่วนนี้นำเสนอการศึกษาเชิงเปรียบเทียบที่ครอบคลุมสำหรับวิธีการ Regression ต่างๆ โดยใช้ชุดข้อมูลเดียวกัน เพื่อให้เข้าใจถึงจุดแข็งและจุดอ่อนของแต่ละเทคนิค

7.8.1 กรณีศึกษา – ชุดข้อมูล Diabetes

ใช้ชุดข้อมูล Diabetes จาก Scikit-learn ซึ่งมี 442 ตัวอย่าง 10 Feature (age, sex, bmi, bp, s1-s6) และค่าเป้าหมายเป็นตัวเลขต่อเนื่องแทนความก้าวหน้าของโรคเบาหวาน

```
from sklearn.datasets import load_diabetes
from sklearn.ensemble import
RandomForestRegressor,
GradientBoostingRegressor, StackingRegressor
from sklearn.model_selection import
cross_val_score
data = load_diabetes()
X, y = data.data, data.target
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=42)
```

7.8.1.1 Simple Linear Regression

Training $R^2 = 0.528$, Testing $R^2 = 0.453$

7.8.1.2 Polynomial Linear Regression

degree=2: Training $R^2 = 0.606$, Testing $R^2 = 0.416$ / degree=3: Overfit (Testing $R^2 = -15.5!$) — degree=2 ดีที่สุด

7.8.1.3 Polynomial + Regularization

Ridge best (alpha=0.001): Testing $R^2 = 0.511$ / Lasso best (alpha=0.001): Testing $R^2 = 0.501$ / ElasticNet best (alpha=0.1, l1_ratio=0.2): Testing $R^2 = 0.467$ — Ridge ให้ผลดีที่สุดสำหรับชุดข้อมูลนี้

7.8.1.4 Multivariable Regression

Training $R^2 = 0.528$, Testing $R^2 = 0.453$ — เท่ากับ Simple Linear Regression

7.8.1.5 Cross-Validation

3-fold $R^2 = 0.489$ / 5-fold $R^2 = 0.482$ / 10-fold $R^2 = 0.462$ — ทุกแบบให้ผลใกล้เคียงกัน

7.8.1.6 Ensemble Methods

ใช้ชุดข้อมูล Diabetes ทดสอบ Random Forest, Gradient Boosting และ Stacking:

```
from sklearn.ensemble import
(RandomForestRegressor,
 GradientBoostingRegressor,
 StackingRegressor)
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
```

```
scaler_d = StandardScaler()
X_train_d = scaler_d.fit_transform(X_train)
X_test_d = scaler_d.transform(X_test)

# --- Random Forest (Bagging) ---
rf = RandomForestRegressor(n_estimators=100,
random_state=42)
rf.fit(X_train_d, y_train)
print("Random Forest Train R²:",
rf.score(X_train_d, y_train)) # 0.921
print("Random Forest Test R²:",
rf.score(X_test_d, y_test)) # 0.467

# --- Gradient Boosting (Boosting) ---
gb = GradientBoostingRegressor(n_estimators=200,
learning_rate=0.05,
max_depth=3, random_state=42)
gb.fit(X_train_d, y_train)
print("Gradient Boost Train R²:",
gb.score(X_train_d, y_train)) # 0.836
print("Gradient Boost Test R²:",
gb.score(X_test_d, y_test)) # 0.453

# --- Stacking (RF + GB + Ridge meta-learner) ---
estimators = [
RandomForestRegressor(n_estimators=100,
random_state=42)],
```

```
("gb",
GradientBoostingRegressor(n_estimators=200,
learning_rate=0.05,
max_depth=3,
random_state=42)) ]
stk = StackingRegressor(estimators=estimators,
final_estimator=Ridge(),
cv=5)
stk.fit(X_train_d, y_train)
print("Stacking Train R2:", stk.score(X_train_d,
y_train)) # 0.526
print("Stacking Test R2:", stk.score(X_test_d,
y_test)) # 0.455
```

ผลลัพธ์: Random Forest (Bagging) Training $R^2 = 0.921$, Testing $R^2 = 0.467$ / Gradient Boosting (Boosting) Training $R^2 = 0.836$, Testing $R^2 = 0.453$ / Stacking Training $R^2 = 0.526$, Testing $R^2 = 0.455$ — สำหรับชุดข้อมูล Diabetes Polynomial + Ridge ยังให้ Testing R^2 สูงสุด (0.511) ชี้ว่าข้อมูลนี้มี Linear Pattern เป็นหลัก

7.8.1.7 สรุปการเปรียบเทียบ

สำหรับชุดข้อมูล Diabetes: Polynomial + Ridge ($\alpha=0.001$) ให้ Testing R^2 สูงสุดที่ 0.511 ตามด้วย Polynomial (degree=2) ที่ 0.416 Ensemble Methods มี Training R^2 สูงมาก (Random Forest 0.921) แต่ Testing R^2 ไม่ได้ดีกว่า Regularization ชี้ให้เห็นว่าการเลือกโมเดลที่ดีที่สุดขึ้นอยู่กับลักษณะของชุดข้อมูล

ตัวอย่างการประยุกต์ใช้งานจากงานวิจัย

สาธิตวุฒิและคณะ (2025) ในงานวิจัยเรื่อง "Enhancing Tourist Forecasting in Thailand's National Parks with Zero-Inflated Models" ได้เปรียบเทียบแบบจำลองการถดถอยสำหรับข้อมูลเชิงนับ (Count Data) จำนวน 4 ประเภท ได้แก่

Poisson Regression, Negative Binomial (NB) Regression, Zero-Inflated Poisson (ZIP) และ Zero-Inflated Negative Binomial (ZINB) โดยใช้ข้อมูลจำนวนนักท่องเที่ยวที่เดินทางเข้าอุทยานแห่งชาติ 146 แห่งในประเทศไทยระหว่างปี พ.ศ. 2559–2565 (จำนวน 12,264 การสังเกต) ผลการวิจัยพบว่า ZINB มีค่า AIC ต่ำสุด (202,669) และ RMSE ต่ำสุด (16,616.12) เมื่อเทียบกับ NB (RMSE = 25,528.13) เนื่องจากข้อมูลมีลักษณะ Overdispersion และ Excess Zeros ซึ่งโมเดล ZINB สามารถจัดการได้พร้อมกัน

นอกจากนี้ ผาพันธ์และคณะ (2023) ในงานวิจัยเรื่อง "Comparison of the Effectiveness of Regression Models for the Number of Road Accident Injuries" ได้นำแบบจำลองการถดถอยมาเปรียบเทียบประสิทธิภาพในการพยากรณ์จำนวนผู้บาดเจ็บจากอุบัติเหตุทางถนน ซึ่งเป็นอีกตัวอย่างของการประยุกต์ใช้ Regression กับข้อมูลสาธารณสุขและความปลอดภัย สะท้อนให้เห็นความหลากหลายของการนำแบบจำลองการถดถอยไปใช้ในบริบทที่แตกต่างกัน

เอกสารอ้างอิง

1. Sathitvudh, S., Wongwiwat, P., & Phaphan, W. (2025). Enhancing tourist forecasting in Thailand's national parks with zero-inflated models. *WSEAS Transactions on Environment and Development*, 21, 284–292. <https://doi.org/10.37394/232015.2025.21.25>
2. Phaphan, W., Sangnuch, N., & Piladaeng, J. (2023). Comparison of the effectiveness of regression models for the number of road accident injuries. *Science & Technology Asia*, 28(4), 54–66.